

Agentic Development of Numerical Software

A practitioner's report from a small-team campaign

Raul Llamas-Sandin
Universidad de Sevilla

May 28, 2026

Abstract

This report describes how a one-developer team can deliver multiple serious numerical-software products in a single year by combining agentic AI tooling with a disciplined workspace. It documents the workspace shape, the philosophy that guided decisions, prompt patterns that worked, the role of skills, hooks, and the harness classifier, and a concrete technology stack centred on Julia for solver cores and JavaScript with Babylon.js for client-side post-processing. It ends with recommendations for any team transitioning to agentic methods on this class of software. The material is based on the JFEM / OpenJFEM, OpenLUDWIG, HEXATOP, OpenFLIGHT, JDATCOM, JAVL, and JXFoil campaigns conducted between 2025 and 2026.

Contents

1	Introduction	4
2	The Workspace	4
2.1	Four worlds, separated by design	4
2.2	The shape on disk	4
2.3	The public/private boundary, enforced three ways	5
2.4	The auto-publish chain	5
2.5	Run-outputs taxonomy	6
3	Philosophy of Working	6
3.1	Direction, not typing	6
3.2	Punctuated work, parallel sessions	6
3.3	Pushing back at the right level	7
3.4	Bipolar models	7
3.5	Fresh sessions when stuck	7
3.6	The human cost is real	8
4	Useful Prompts	8
4.1	The world-class plan	8
4.2	The world-class audit	8

4.3	The adversarial review	8
4.4	The wrong-answer probe	8
4.5	The literature ask	9
4.6	The validation prompt	9
4.7	The MCP prompt	9
4.8	The hand-off prompt	9
5	The Agents We Use	9
5.1	Claude Code – the daily driver	9
5.2	Codex – well-scoped feature work	10
5.3	Antigravity 2.0 – long autonomous arcs	10
5.4	Gemini – chat-style, multimodal	10
5.5	Selection heuristic	10
6	Skills	10
6.1	What a skill is	10
6.2	When you would write a skill	11
6.3	Our five skills	11
6.4	The trigger-phrase art	11
6.5	When NOT to write a skill	11
6.6	The chicken-and-egg solution	12
7	Hooks	12
7.1	What a hook is	12
7.2	When you would write a hook	12
7.3	Our auto-sync hook	12
7.4	When NOT to write a hook	13
7.5	Skills vs. AGENTS.md vs. hooks	13
8	The Classifier as a Safety Net	13
9	The Technology Stack	14
9.1	Julia for solver cores	14
9.2	JavaScript for front-ends	14
9.3	msgpack with binary JSON payloads	15
9.4	Babylon.js as 3D engine	15
9.5	What we would re-do differently	15
10	Validation Discipline	15

10.1 The four-artifact rule	15
10.2 External vs. synthetic validation	16
10.3 TDD as guard rail	16
10.4 EULA compliance	16
11 Documentation Build Chain	16
11.1 LaTeX for shipped documents	16
11.2 Markdown for working notes	16
11.3 CHANGELOG discipline	16
11.4 PowerPoint companions	17
12 Recommendations	17

1 Introduction

Numerical software has long been an outlier of software engineering. A correctness regression is invisible until somebody compares two numbers, reference data is often larger than the code, validated test sets cannot always be shared, and a useful failed experiment is frequently more valuable than a passing unit test. None of the usual app-development playbooks transfer cleanly.

Agentic AI changes the picture, but does not lift the constraints. It removes the typing bottleneck and lowers the cost of exploration; it does not, by itself, fix the discipline problem. If anything, the combination of a fast agent and a careless workspace accelerates contamination of the code base with throwaway scripts, half-validated results, and orphaned experiments.

This report describes the way of working we converged on, after eighteen months of campaigning, to keep the discipline while keeping the speed. It is opinionated, narrow in scope (numerical solvers, not generic web software), and specific about tooling. Where we recommend a choice, the recommendation is grounded in something we actually shipped.

The intended reader is a small team – typically one to three engineers – working on numerical software with a strong validation requirement and an existing tail of in-house code. The report assumes familiarity with the practical use of an agentic CLI such as Claude Code, Codex, Gemini CLI, or Antigravity, and with at least one numerical language (Julia, Python, C++, or FORTRAN).

2 The Workspace

2.1 Four worlds, separated by design

Our workspace separates four worlds that ordinary repositories tend to conflate:

1. **Public source code**, fit to publish on GitHub.
2. **Development evidence**: validation decks, run outputs, campaign notes, diagnostic artifacts.
3. **External tools and reference solvers**, including commercial payloads and third-party source.
4. **Documentation sources in work**: LaTeX, Beamer, figures, talks, pitches, anything from which we generate a PDF.

Keeping these separate prevents recurring failures: accidentally committing private solver binaries or huge result files; losing the exact validation case that produced a number; mixing generated results with source code; letting agent sessions leave scattered notes with no living summary; and – the failure that prompted our last major restructure – treating "I have a PDF I want to share" as a reason to drop it into the public repository folder regardless of what it contains.

2.2 The shape on disk

Figure 1 shows the workspace tree. The numbered top-level folders are deliberate: they put the public folder first, the development evidence second, the external tools third, and the documentation-in-work fourth. The order matters because it is the order in which an agent walking in cold should read.

```

PROJECT_WORKSPACE/
|-- WORKSPACE_MAP.md
|-- AGENTS.md                                <-- standing rules
|-- .claude/
|   |-- skills/                              <-- procedural recipes
|   |-- settings.json                        <-- hooks
|   '-- settings.local.json                 <-- per-machine permissions
|-- 01_PROJECT_FOLDER/
|   '-- <Public_GitHub_repo>/<Project>/      <-- ONLY folder pushed to GitHub
|       '-- Reference_documentation/        <-- auto-receives all PDFs from 04
|-- 02_DEVELOPMENT_AND_VALIDATION/
|   |-- Development_campaign/               <-- STATUS_AND_ROADMAP.md = truth
|   |-- VALIDATION_FILES/                  <-- input decks + registry
|   |-- RUN_OUTPUTS/                       <-- timestamped run reports
|   |-- DEVELOPMENT_TOOLS/                 <-- helper scripts, not for GitHub
|   |-- External_papers/                   <-- ArXiv, GitHub clones, etc.
|   '-- Workspace_definition_and_setup/    <-- meta documentation
|-- 03_EXTERNAL_TOOLS/                     <-- Nastran, MYSTRAN, TACS, etc.
'-- 04_DOCUMENTATION_IN_WORK/
    |-- sync_pdfs_to_reference.ps1          <-- THE auto-sync engine
    |-- build_all.ps1                      <-- builds every .tex, then syncs
    |-- PROJECT_DOCS_IN_WORK/               <-- the project's own LaTeX docs
    |-- NN_<TalkName>/                     <-- one folder per external talk

```

Figure 1: Workspace layout used throughout this report.

2.3 The public/private boundary, enforced three ways

The only folder that ever reaches GitHub is `01_PROJECT_FOLDER/<Public_GitHub_repo>/<Project>/`. `git status`, `git add`, `git commit`, and `git push` run only from there. The boundary is enforced at three layers, in order of how often each one catches a mistake:

1. **Three .gitignore guards:** one at the workspace root, one at `01_PROJECT_FOLDER/`, and one inside the public package. Each one allowlists only the path below it, so a stray `git init` higher up the tree finds no files to commit.
2. **AGENTS.md:** a standing rules file that every agent loads at session start. It states the boundary, describes the allowed exceptions (the auto-synced PDFs), and lists things an agent should refuse to do.
3. **Skills:** procedural recipes that the agent invokes when the user's intent matches. The `update-github-repo` skill in particular checks `cwd`, refuses to stage files from private folders, and requires a `CHANGELOG` entry.

2.4 The auto-publish chain

Figure 2 shows the flow that turns a LaTeX source under `04_DOCUMENTATION_IN_WORK/` into a published PDF on `origin/main`.

The sync engine is a single PowerShell script. It is source-driven (it copies from `04_DOCUMENTATION_IN_WORK/` into the public folder, never the other way), idempotent (only copies if the source is newer), and reports filename collisions so that two PDFs in different subfolders cannot silently overwrite each other.

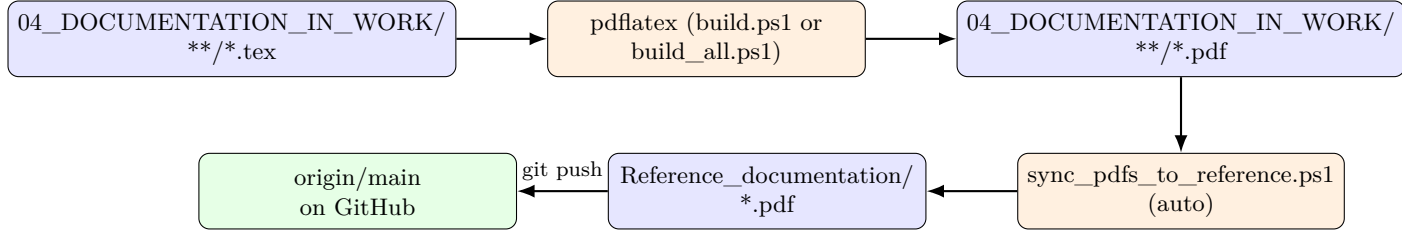


Figure 2: The auto-publish chain. The sync step is the only one that crosses the public/private boundary, and it does so as a controlled, idempotent, source-driven mirror.

2.5 Run-outputs taxonomy

Generated run outputs grow faster than any other category. We organise them by solution family and calendar quarter:

`RUN_OUTPUTS/<SOL_family>/<YYYYQn>/<RUN_TAG>/<YYYYMMDD_HHMMSS>/`

A campaign that does not have an obvious solution-family home goes under `PARITY/` or `DIAGNOSTICS/`. Exploratory runs that do not deserve a registry row live in `SCRATCH_<date>_<tag>/` inside their family folder. This taxonomy emerged from watching the flat layout become unmanageable within three months.

3 Philosophy of Working

3.1 Direction, not typing

The phrase that captures the largest single shift in how we work is “*Do not touch the code or the documentation; agents do everything, focus on direction and intuition at a high level of abstraction.*” We treat this as a rule, not a suggestion. The consequences are concrete:

- No hand-edited code lives in the repository. If a one-line tweak is needed, it is requested from the agent so the change is documented in conversation history.
- No hand-edited documentation lives anywhere. Documents are regenerated, not patched.
- Strategic decisions are the user’s domain; tactical execution is the agent’s domain.

The point is not purity. It is that hand-edits introduce drift between the model’s mental picture of the code base and the actual state, and that drift is the most common source of regressions in agentic work.

3.2 Punctuated work, parallel sessions

Agentic development is punctuated, not continuous. A turn typically looks like: direct the agent (one to five minutes), wait for the agent to think and write (one to five minutes), wait for the code to run (seconds to several minutes), inspect, redirect. The user is free during steps two and three.

We treat the agent’s wall clock as our scheduling unit. During the wait windows we run a second session on a different task, answer email, or read a paper we plan to feed back in. Practical limits we observe:

- Two concurrent sessions on related tasks is comfortable.
- Three concurrent sessions across unrelated codebases is the upper bound before context-switching costs eat the savings.
- Long autonomous runs (Antigravity-style) free up real hours but require a longer review pass afterwards.

3.3 Pushing back at the right level

Models default to a confident, slightly arrogant tone. When the model proposes the wrong thing, the user must push back, and models thank you for that – their next answer is usually better.

A more recent lesson is to push back at the right *level*. In our workspace there are three layers that can stop a bad action: the user’s own decisions, the agent’s internal logic, and the harness’s classifier. When something is being blocked, the temptation is to override the nearest layer. The correct move is almost always to fix it upstream. Examples:

- A PDF that does not belong in the public repository is in `Reference_documentation/`. The correct fix is to move or delete it from the source, not to override the classifier at `git push`.
- A planning prompt is producing waffle. The correct fix is to rewrite the prompt with explicit constraints, not to manually rewrite the agent’s output.
- A skill is not triggering. The correct fix is to widen the description’s trigger phrases, not to invoke the body manually.

3.4 Bipolar models

The model alternates between over-confidence and surrender. Over a campaign, we saw both modes in roughly equal proportion:

- **Over-confidence:** proposes a fix with no caveats when the fix has obvious failure modes. Counter by asking “what would make this wrong?”.
- **Surrender:** announces that the problem cannot be solved when an obvious avenue is still open. Counter by listing the avenues you can see and asking the model which one is wrong.

Both modes are common around plateaus. The plateau itself is rarely solved by “trying harder”; it is usually solved by feeding new material (a paper, a reference implementation, a fresh diagram).

3.5 Fresh sessions when stuck

When a session drifts off course, we start a new one rather than try to salvage. Hand-off material (`STATUS_AND_ROADMAP.md`, `NEXT_SESSION_<date>.md`, the validation registry) makes this cheap. The cost of forcing a stuck session to recover is almost always higher than the cost of catching the new session up.

3.6 The human cost is real

The widely repeated promise that AI lets us work less has not matched our experience. We produce far more in the same time, but the bottleneck moves from typing to deciding, and deciding is harder. Burn-out is a real risk. Planning for cognitive recovery – shorter focused blocks, hard stops, real breaks – matters more, not less, when working agentially.

4 Useful Prompts

The single most undervalued asset in agentic work is a library of prompts that have demonstrably worked. Below are patterns we use repeatedly. Each is given in the form we actually paste, not in an idealised version.

4.1 The world-class plan

Lay out a plan to develop a world-class X solver. Tell me what papers and repositories you would like to read. Store the plan in memory.

This prompt does three things at once: it forces the model to commit to a scope, it surfaces the model’s own picture of the relevant literature, and it leaves a planning artifact that subsequent sessions can resume.

4.2 The world-class audit

Look at the current state of the X solver. What is left to claim world class? Be specific. What is the next single thing to address?

Periodic use of this prompt at plateaus gets the model to stop, reflect, and re-prioritize. It almost always surfaces an angle the user had not thought to ask about.

4.3 The adversarial review

Read the attached plan as if you were the most sceptical reviewer in the field. What is wrong with it? What is hand-waved? What assumption would the reviewer reject first?

Use this on a plan the agent itself just wrote. The model has no trouble attacking its own output when explicitly asked.

4.4 The wrong-answer probe

Assume your last answer is wrong. What is the single most plausible reason? Propose the corrected answer.

A surprisingly effective response to over-confident output. It also works on numerical results that “feel” wrong without obvious evidence.

4.5 The literature ask

Before you propose a solution, tell me what specific references – papers, books, code repositories – you would most want to read for this problem, and what you would learn from each. I will feed you what I can find.

This converts a stalled agent into a research assistant. Models are often better at naming the literature than at solving the problem with no input.

4.6 The validation prompt

Add a validation case for X. Follow the four-artifact rule. Store the deck in `VALIDATION_FILES`, add a row to `VALIDATION_CASE_REGISTRY.csv`, run it under `RUN_OUTPUTS/<SOL>/<YYYYQn>/`, and write a `RUN_REPORT.md`.

This prompt triggers the `add-validation-case` skill (Section 6) and produces the full evidence trail in one turn.

4.7 The MCP prompt

Trim the aircraft at 200 kt, 5000 m, 1.5 g. Then sweep the elevator deflection and plot the response.

When the project exposes an MCP, the natural-language version of a control sequence is the API. We use this in OpenFLIGHT for trim and control studies, and in HEXATOP for verbal definition of the optimization domain.

4.8 The hand-off prompt

The session is about to end. Update `STATUS_AND_ROADMAP.md` with what changed today. Write a `NEXT_SESSION_<date>.md` with the three highest-priority items for the next session. Update `AGENT_MEMORY.md` with anything worth remembering.

End every working session with this. The cost is two minutes; the benefit is that the next session, possibly on a different tool, starts from a known state.

5 The Agents We Use

We deliberately use several agents and switch between them. No single agent is best at everything we do.

5.1 Claude Code – the daily driver

Strengths: conversational, strong on multi-file refactors, mature CLI plus VS Code extension, file-based memory and workspace-local skills, predictable per-token cost. We use Claude Code as the default for mixed development, debugging, and documentation work.

Weaknesses: best when the user provides direction; less productive on extremely long autonomous arcs than dedicated autonomous agents.

5.2 Codex – well-scoped feature work

Strengths: tight task-with-verifier loop, strong IDE integration, good on a clearly framed feature with explicit acceptance criteria.

Weaknesses: needs the scope to be tighter than the daily-driver style; less effective when the request is exploratory.

5.3 Antigravity 2.0 – long autonomous arcs

Strengths: runs unattended for substantial periods; produces a useful trace of decisions; appropriate when the user can specify an endpoint but not the route. We use it for large refactors and for filling in breadth (test coverage, doc passes).

Weaknesses: per-run cost can be heavy; the review pass on the output is non-trivial.

5.4 Gemini – chat-style, multimodal

Strengths: chat-mode iteration, strong multimodal handling (images, PDFs). We used Gemini in chat mode to build the universal FPO visualizer until that workflow hit its ceiling.

Weaknesses: chat-only workflows do not scale beyond a certain code-base size; need to be paired with a CLI agent for serious campaigns.

5.5 Selection heuristic

A rough heuristic we use:

- Mixed development, debugging, doc work, or anything that benefits from conversation: **Claude Code**.
- A clearly defined feature with verifiable acceptance criteria: **Codex**.
- An open-ended arc with a clear endpoint: **Antigravity**.
- Visual / multimodal work or a quick research iteration: **Gemini**.

We avoid collecting tools. Two daily-driver agents and one specialist is the upper bound that has worked for us.

6 Skills

6.1 What a skill is

A skill is a procedural recipe living at `.claude/skills/<skill-name>/SKILL.md`. The file has YAML frontmatter (name, description) and a body of instructions. The agent loads the names and descriptions at session start; the body is loaded only when the skill is actually invoked.

This matters: skills cost zero context when idle. We currently have five skills and the overhead is invisible.

6.2 When you would write a skill

Write a skill when a procedure has all three of these properties:

1. It is requested often enough that re-typing the instructions is wasteful.
2. It has discipline that is easy to skip under pressure (steps that protect the user from themselves).
3. It has refusal conditions that are easier to enforce procedurally than to remember.

6.3 Our five skills

- **add-validation-case** – enforces the four-artifact rule (deck, registry row, timestamped run folder, RUN_REPORT.md).
- **publish-doc** – runs the right build script, lets the sync mirror PDFs into the public repository, adds a CHANGELOG entry.
- **update-github-repo** – runs the sync, performs public-folder cwd and private-artifact checks, requires a CHANGELOG entry for non-trivial changes, stages by explicit path, refuses force-push to main.
- **bootstrap-workspace** – creates a fresh workspace following this report’s shape, verifies the toolchain, copies the skill set across.
- **write-latex-doc** – codifies the LaTeX conventions in the workspace (folder taxonomy, naming, theme, TikZ-only rule, build pattern).

6.4 The trigger-phrase art

A skill fires when the user’s request matches its **description:** field. Writing a good description is the highest-leverage part of authoring a skill. Two rules that consistently produced useful triggers:

1. Include concrete example phrases the user might actually type (“rebuild the docs”, “ship the user guide update”).
2. Avoid abstract category labels (“documentation workflow” triggers nothing).

The phrases are searched verbatim against natural-language input; the closer the phrase matches the user’s habitual wording, the more reliable the trigger.

6.5 When NOT to write a skill

- The procedure runs once a quarter. The description rotting in the loader costs more than the saving.
- It depends on judgment that varies session to session. Skills bind to procedure, not to taste.
- It is a hard guard rather than a recipe. That is what a hook is for.

6.6 The chicken-and-egg solution

The `bootstrap-workspace` skill lives inside an existing workspace, which makes it useless on a fresh machine. We bridge the gap with `bootstrap-from-zero.ps1`: a single PowerShell file kept in the developer's dotfiles or sync folder that fetches the skill set from a local path, a Git URL, or an HTTPS zip URL, seeds the minimal `.claude/skills/` tree at a new target, and hands off to `/bootstrap-workspace`.

7 Hooks

7.1 What a hook is

A hook is a shell command that the harness fires automatically in response to a tool event. Hooks live in `.claude/settings.json` as JSON-configured matchers and commands. The most useful event types in our context are `PostToolUse` (after every tool call), `Stop` (after the agent stops), and `SessionStart`.

A hook is the only mechanism that runs without the agent's participation. That property is the entire reason hooks exist.

7.2 When you would write a hook

Write a hook when the agent cannot reliably do the action itself, or when you want the action to happen even if the agent forgets. The two patterns we have used productively:

1. **Automatic mirror.** A `PostToolUse` hook that runs after every shell call, invokes our PDF sync, and keeps `Reference_documentation/` fresh without us having to remember.
2. **Hard boundary guard.** A `PreToolUse` hook that inspects the command about to run and refuses if it would violate the public/private boundary. We considered but have not yet deployed this one.

7.3 Our auto-sync hook

The hook below is configured in `.claude/settings.json`. After any Bash or PowerShell tool call it invokes the sync script in `-Quiet` mode. The script is idempotent and fast on a no-change run, so triggering it on every shell call is cheap.

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Bash|PowerShell",
        "hooks": [
          {
            "type": "command",
            "command": "powershell -NoProfile -ExecutionPolicy Bypass -File \"${CLAUDE_PROJECT_DIR}\\04_DOCUMENTATION_IN_WORK\\sync_pdfs_to_reference.ps1\" -Quiet"
          }
        ]
      }
    ]
  }
}
```

7.4 When NOT to write a hook

- The action is needed only when the user asks for it. That is a skill.
- The action would fire too often and become wallpaper. A **Stop** hook that nags about updating `STATUS_AND_ROADMAP.md` on every turn is an example we considered and rejected.
- The action depends on judgment about what changed. Skills are better at that.

7.5 Skills vs. AGENTS.md vs. hooks

The three mechanisms cover overlapping but distinct ground:

- **AGENTS.md** – always-loaded standing rules. Inform, do not enforce. Free in context terms but invisible to the harness.
- **Skills** – procedures triggered by intent. Loaded only when invoked. Carry checklists and refusals. They run at the agent’s level: the agent chooses to apply them.
- **Hooks** – hard guards on tool calls. Loaded by the harness. They run *regardless* of what the agent intends.

A rough decision tree: if the rule is a standing fact, write it in AGENTS.md. If the rule needs to fire when the user asks for something specific, write a skill. If the rule must hold even when the user and agent agree to skip it, write a hook.

8 The Classifier as a Safety Net

A property of Claude Code we did not appreciate at first is that the harness contains its own classifier that can refuse a tool call independently of the agent’s reasoning. Multiple times during the campaign described in this report the classifier blocked an action that the user had asked for and the agent was willing to perform.

In every case, on review, the classifier was right. Examples:

- The user asked to push three PowerPoint duplicates that had ended up in the public repository folder. The agent identified the contradiction with the workspace rules but, after the user confirmed, was prepared to proceed. The classifier refused on the boundary grounds. The fix was to move the files out of the public folder, not to override the classifier.
- The user asked to push directly to `main`. The classifier flagged that the workspace’s own `CONTRIBUTING.md` recommended feature branches. The eventual resolution was an explicit permission rule, but the warning was correct.
- A bulk move of run-outputs folders was refused as “not specifically authorized”. The agent reframed as a reviewable script the user could execute, which was a better outcome.

The lesson is to treat the classifier as a useful third opinion, not as an obstacle. When it refuses, the right move is usually to ask whether the rule it is enforcing should actually be relaxed, and if so to relax it explicitly (a permission rule in `settings.local.json`, a change to AGENTS.md), rather than to find a way around.

9 The Technology Stack

9.1 Julia for solver cores

The single largest reason we chose Julia is the absence of a two-language barrier. Glue code and inner loops are the same language. Type inference and LLVM produce code that runs at C speed for the hot paths, while staying readable for the agent.

Concrete benefits we observed across the campaigns:

- Native GPU paths via `CUDA.jl`, `AMDGPU.jl`, `oneAPI.jl`. Same array abstraction across backends. `OpenLUDWIG` and `HEXATOP` both exploit this.
- Composable AD (`ForwardDiff`, `Zygote`, `Enzyme`) works on generic code. JFEM’s adjoint sensitivity machinery for static and buckling eigenvalues is built on this.
- Strong scientific-computing ecosystem (`HDF5`, `KrylovKit`, `IterativeSolvers`, `AlgebraicMulti-grid`, `WriteVTK`) so the solver author rarely re-implements numerical primitives.
- Agent legibility: short modules, no header/source split, REPL-driven iteration. The agent reads and modifies real code, not a wrapper layer.

What we would warn against:

- “Python with a Julia kernel” patterns. They re-introduce the two-language tax we left to escape.
- Custom binary formats for solver IO. Use `HDF5` and `JSON`; both round-trip cleanly to the JavaScript client.

9.2 JavaScript for front-ends

For any user-facing interaction we ship a pure-JavaScript front-end. The reasons are unglamorous but decisive:

- The browser is the most widely installed runtime that exists.
- No installation step for the end user.
- `Babylon.js` as 3D engine. `WebGPU` when available, `WebGL` fallback, no platform-specific build.
- `p5.js` for quick 2D sketches; `three.js` as a `Babylon` alternative on lighter scenes.
- Tooling cost is low: no build step is required during development; a single bundle for shipping.

The agent writes JavaScript well, especially for visual code.

9.3 msgpack with binary JSON payloads

We use msgpack as the transport between Julia server and JavaScript client. The reasons are:

- Compact wire format; small per-message overhead.
- Both ends have first-class libraries.
- Binary JSON payloads inside msgpack frames give us a schema-light contract that survives schema drift better than a fixed Protobuf IDL.

We avoid REST gymnastics. The protocol is one long-lived socket and a dispatch on message type.

9.4 Babylon.js as 3D engine

Babylon.js was chosen for its WebGPU support, mature picking and selection, scene-graph performance on tens of thousands of meshes, and its TypeScript declaration completeness for agent legibility. The post-processors for both JFEM and OpenLUDWIG share the same loader and scene plumbing.

9.5 What we would re-do differently

If we restarted, we would:

- Adopt the four-folder workspace from the very first commit, not after a year of campaigning into a flat layout.
- Write the validation-case registry skill before the first validation case, not after fifty.
- Decide the LaTeX-vs-Markdown allocation on day one and stick to it (LaTeX for things that go to PDF, Markdown for everything else).

10 Validation Discipline

10.1 The four-artifact rule

Every validation case in our workspace must produce four artifacts:

1. A stable input deck under `VALIDATION_FILES/`.
2. A row in `VALIDATION_CASE_REGISTRY.csv`.
3. A timestamped run folder under `RUN_OUTPUTS/<SOL>/<YYYYQn>/`.
4. A `RUN_REPORT.md` with inputs, command, reference, tolerance, headline result, verdict, interpretation.

A case that does not produce all four is exploratory. It belongs in a `SCRATCH_<date>_<tag>/` folder, not as a real registry entry. The `add-validation-case` skill exists to enforce this rule.

10.2 External vs. synthetic validation

We rigorously separate validation cases sourced from external reference solvers (Nastran, MYSTRAN, TACS, XFOIL, OpenLB, PALABOS, AVL, DATCOM, analytical solutions) from synthetic cases produced by the agent. Conclusions drawn from synthetic cases are tautological. The folder structure makes the distinction immediately visible.

10.3 TDD as guard rail

For solver work we use test-driven development as a discipline against silent regression. The tests are unit-level (kernels, BC handling, constraint elimination) and functional (full-pipeline runs against a small canonical deck). Adding a new feature never lands without a test that would have failed before.

10.4 EULA compliance

When developing code that needs to match a commercial reference (e.g. Nastran) we do not violate the EULA. We use academic results and public repositories. We do not run the commercial solver to generate matching data. This is a discipline issue and the workspace structure (`03_EXTERNAL_TOOLS/`) makes it visible – commercial payloads sit there explicitly, are never auto-loaded, and never inform a synthetic test.

11 Documentation Build Chain

11.1 LaTeX for shipped documents

All shipped documents are LaTeX. The reasons are practical: LaTeX produces references in a stable form (DOIs, hyperlinks, figure references), TikZ produces self-contained figures that require no binary assets, Beamer keeps slides consistent across talks, and the output is a single PDF that a user can read with no tooling.

We avoid binary image includes; figures are TikZ wherever possible. The exceptions – the few real screenshots and renders – are flagged as illustration placeholders in the source so a draft read-through makes every gap visible.

11.2 Markdown for working notes

Markdown is used for working notes, READMEs, and the campaign log. It is not shipped; it is read in VS Code while the campaign runs.

Mermaid diagrams inside Markdown work well for flow diagrams the team needs but the customer does not. Both Markdown and Mermaid render in place in VS Code with extensions.

11.3 CHANGELOG discipline

Every non-trivial commit to the public repository includes a CHANGELOG entry under `[Unreleased]`. The `update-github-repo` skill refuses to push without one. This produces a single timeline of “what changed when” that the next maintainer (human or agent) can read at any time.

11.4 PowerPoint companions

Some delivery venues require PowerPoint. We produce `.pptx` companions to the LaTeX-built PDFs and ship both. The PowerPoint files live in `Reference_documentation/` alongside the PDFs and travel with the repository on push.

If the PowerPoint files become a regular thing, the right next step is Git LFS for `*.pptx` so the binary content stays out of the main pack file.

12 Recommendations

A condensed list, in priority order, of what we would tell a small team starting a numerical-software campaign today:

1. **Treat the workspace as the primary asset.** The shape of the workspace determines whether the team's velocity compounds or decays.
2. **Establish the four-folder split from the first commit.** Public code, development evidence, external tools, documentation in work. Number the folders, separate the worlds, do not let them mix.
3. **Pick one daily-driver agent and at most two specialists.** Do not collect tools.
4. **Adopt the four-artifact validation rule on day one.** Stable deck, registry row, timestamped run folder, run report. The `add-validation-case` skill is the cheapest way to enforce it.
5. **Spend the first month on infrastructure, not features.** Skills, hooks, `AGENTS.md`, validation taxonomy, documentation build chain. These compound; features without them do not.
6. **Use `AGENTS.md` for standing rules, skills for procedures, hooks for guards.** Pick the right mechanism for each rule.
7. **Write the trigger phrases for skills the way you actually speak.** Abstract category labels do not trigger.
8. **Treat the classifier as a useful third opinion.** When it refuses, look upstream first; relax the rule explicitly if needed, never silently.
9. **Push back at the right level.** If something is wrong, fix it at the source, not at the next layer that complains.
10. **Separate external from synthetic validation.** Conclusions drawn from agent-generated data are tautological.
11. **Auto-generate every document.** Hand edits drift.
12. **Choose Julia for solver cores.** The two-language tax is the single largest cost the alternative pays.
13. **Choose JavaScript with Babylon.js for the front-end.** The browser is the only runtime universally installed.
14. **Use msgpack with binary JSON for transport.** Resist the urge to invent a custom protocol.

15. **End every session with a hand-off note.** Two minutes of cost; the next session, possibly on a different tool, starts on solid ground.
16. **Invest in local GPU as soon as the workload justifies it.** Privacy, cost predictability, offline use, and a path to fine-tune on your own code base.
17. **Plan for the human cost.** Agentic work is more productive and more demanding. Real breaks, hard stops, shorter focused blocks.

Acknowledgement

This report is itself a product of the workflow it describes. The text was drafted with an agent, reviewed and corrected by the author, and built through the chain documented in Section 11. The underlying campaigns (JFEM, OpenLUDWIG, HEXATOP, OpenFLIGHT, JDATCOM, JAVL, JXFoil) were carried out at Universidad de Sevilla.